Studio Nx

JS 302 — Sampler 연결

2026-10-21

JavaScript 302 — Sampler 연결 (`instruments.js`, `session.js`, `offline.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/instruments.js`, `studionx/frontend/static/js/audio/session.js`, `studionx/frontend/static/js/audio/offline.js`
작업	S1-11c Sampler MIDI 연주: 악기 층에서 Sampler를 울리고, 재생과 내보내기가 Sampler의 샘플을 미리 불러옵니다 (FR-241)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 301 Sampler 음원, 다음 303 샘플 고르기

1. 하는 일

301의 `playSamplerNote`가 소리를 내려면 두 가지가 더 필요합니다. 하나는 트랙의 악기 층이 Sampler를 알아보고 그 함수를 부르는 것이고, 다른 하나는 노트가 울리기 전에 샘플이 디코딩되어 있는 것입니다. 이 문서는 그 연결을 설명합니다.

```
노트 → NotePlayer → InstrumentRack.playNote(트랙, 음높이, ...)
                      각 악기 층마다
                        plugin_code === 'sampler' ─▶ getBuffer(params.sample_id) 로 샘플을 찾아 playSamplerNote
                        그 밖 (synth_sub)         ─▶ playSynthNote

재생 시작: PlaybackSession.start        ┐ 클립의 샘플 + samplerSampleIds 로 구한
내보내기:  renderOffline                ┘ Sampler 의 샘플을 함께 디코딩
```

2. 언제 쓰나

쓰는 곳	하는 일
`PlaybackSession._ensure` (290)	`InstrumentRack`을 만들 때 세 번째 뒤에 `getBuffer`(재생 세션이 담아 둔 샘플)를 넘깁니다.
`PlaybackSession.start` (290)	오디오 클립의 샘플과 Sampler의 샘플을 함께 불러온 뒤에 재생을 시작합니다.
`renderOffline` (299)	같은 방식으로 Sampler의 샘플을 디코딩하고, `InstrumentRack`에 `getBuffer`를 넘깁니다.

3. 코드

3.1 악기 층 (`instruments.js`)

```
export class InstrumentRack {
  /**
   * @param {BaseAudioContext} context
   * @param {function(number|string):AudioNode} trackInput  트랙 번호 → 입력 노드 (MixerGraph.trackInput)
   * @param {function(number|string):Array<object>} getInstruments  트랙 번호 → 악기 층 목록
   *   층: { id, plugin_code, schema, params, volume_db, pan, is_enabled }
   * @param {function(number|string):(AudioBuffer|null)} [getBuffer]  샘플 번호 → 디코딩된 샘플 (Sampler가 씀)
```

```
   */
  constructor(context, trackInput, getInstruments, getBuffer) {
    this.context = context;
    this.trackInput = trackInput;
    this.getInstruments = getInstruments;
    this.getBuffer = getBuffer ? getBuffer : function () { return null; };
    this.layers = {};   // 층 번호 → { gain, pan }
  }

  /** 층의 출력 노드를 (없으면) 만들고 볼륨 · 팬을 맞춤 */
  _layer(trackId, inst) {
    const key = String(trackId) + ':' + String(inst.id);
    const ctx = this.context;
    let layer = this.layers[key];
    if (!layer) {
      const gain = createStereoGain(ctx);
      const pan = typeof ctx.createStereoPanner === 'function' ? ctx.createStereoPanner() : null;
      if (pan) { gain.connect(pan); pan.connect(this.trackInput(trackId)); }
      else gain.connect(this.trackInput(trackId));
      layer = { gain: gain, pan: pan };
      this.layers[key] = layer;
    }
    layer.gain.gain.value = layerGain(inst.volume_db);
    if (layer.pan) layer.pan.pan.value = Math.max(-1, Math.min(1, Number(inst.pan) || 0));
    return layer;
  }

  /**
   * 노트 하나를 트랙의 모든 악기 층에 울림
   * @returns {Array<{stop:function(number):void}>} 울린 소리 (층마다 하나)
   */
  playNote(trackId, pitch, velocity, when, durationSec) {
    const out = [];
    const self = this;
    (this.getInstruments(trackId) || []).forEach(function (inst) {
      if (inst.is_enabled === false) return;
      if (PLAYABLE_CODES.indexOf(inst.plugin_code) < 0) return;
      const layer = self._layer(trackId, inst);
      const params = resolveParams(inst.schema, inst.params);
      if (inst.plugin_code === 'sampler') {
        const buffer = params.sample_id != null ? self.getBuffer(params.sample_id) : null;
        out.push(playSamplerNote(self.context, layer.gain, params, buffer, pitch, velocity, when, durati
onSec));
      } else {
        out.push(playSynthNote(self.context, layer.gain, params, pitch, velocity, when, durationSec));
      }
    });
    return out;
  }
}
```

3.2 재생 시작 때 샘플 불러오기 (`session.js`)

```
async start(fromTick) {
  this._ensure();
  const self = this;
```

```
  const deps = this.deps;
  this.clips = deps.getClips();
  this.noteEvents = expandNotes(deps.getMidiClips ? deps.getMidiClips() : []);
  const ids = this.clips.filter(function (c) { return c.sample_id != null; }).map(function (c) { return
c.sample_id; });
  // Sampler 악기가 쓰는 샘플도 함께 불러옴 (S1-11c)
  const trackIds = deps.getTracks().map(function (t) { return t.track_id; });
  samplerSampleIds(deps.getInstruments ? deps.getInstruments : function () { return []; }, trackIds)
    .forEach(function (id) { if (ids.indexOf(id) < 0) ids.push(id); });
  await Promise.all(ids.map(function (id) { return self._loadSample(id); }));

  this.mixer.applyTracks(deps.getTracks());
  this.effects.sync(deps.getTracks());
  this.engine.setTempo(deps.getBpm(), deps.getTempoChanges ? deps.getTempoChanges() : []);
  if (deps.getMetronome) this.engine.setMetronome(deps.getMetronome());
  const loop = deps.getLoop ? deps.getLoop() : null;
  this.engine.setLoop(loop ? loop.start : 0, loop ? loop.end : 0, !!loop);

  this.player.stopAll();
  this.notePlayer.stopAll();
  this.lastTick = null;
  this.engine.start({ fromTick: fromTick });
  this.playing = true;
  // 시작 위치에 걸쳐 있는 클립은 중간부터 바로 울림
  const t = this.engine.transport;
  clipsCovering(this.clips, t.originTick).forEach(function (c) {
    self.player.schedule(c, t.originTick, t.originTime, self.engine.transport.baseBpm, self.engine.trans
port.changes);
  });
}
```

3.3 내보내기의 샘플 디코딩 (`offline.js`)

아래 함수 안에서 `samplerSampleIds`로 Sampler의 샘플 번호를 `ids`에 더하는 부분과, `InstrumentRack`을 만들 때 `getBuffer`를 넘기는 한 줄이 이번에 추가되었습니다.

```
export async function renderOffline(o) {
  const changes = o.tempoChanges || [];
  const progress = o.onProgress || function () {};
  const cancelled = o.isCancelled || function () { return false; };
  const Ctx = o.OfflineContext || (typeof OfflineAudioContext !== 'undefined' ? OfflineAudioContext : nu
ll);
  if (!Ctx) throw new Error('이 브라우저는 OfflineAudioContext 를 지원하지 않습니다');

  const rangeSec = ticksToSecondsWithChanges(o.endTick, o.bpm, changes) - ticksToSecondsWithChanges(o.st
artTick, o.bpm, changes);
  const totalSec = rangeSec + Math.max(0, o.tailSec || 0);
  const frames = Math.max(1, Math.ceil(totalSec * o.sampleRate));
  const ctx = new Ctx(2, frames, o.sampleRate);

  const audioClips = clipsInRange(o.audioClips, o.startTick, o.endTick);
  const midiClips = clipsInRange(o.midiClips, o.startTick, o.endTick);

  // 1) 샘플 디코딩 (오프라인 컨텍스트의 샘플레이트로 다시 맞춰짐). 전체 진행률의 앞 10%
  const buffers = {};
```

```
  const ids = [];
  audioClips.forEach(function (c) { if (c.sample_id != null && ids.indexOf(String(c.sample_id)) < 0) id
s.push(String(c.sample_id)); });
  // Sampler 악기가 쓰는 샘플도 함께 디코딩 (S1-11c)
  samplerSampleIds(o.getInstruments, o.tracks.map(function (t) { return t.track_id; })).forEach(function
(id) {
    if (ids.indexOf(String(id)) < 0) ids.push(String(id));
  });
  for (let i = 0; i < ids.length; i++) {
    if (cancelled()) throw new Error('cancelled');
    try {
      const bytes = await o.loadSampleBytes(Number(ids[i]));
      buffers[ids[i]] = await ctx.decodeAudioData(bytes);
    } catch (err) {
      if (typeof console !== 'undefined') console.warn('[export] 샘플 ' + ids[i] + ' 을(를) 불러오지 못해
건너뜁니다', err);
    }
    progress(0.1 * (i + 1) / ids.length);
  }

  // 2) 재생과 같은 그래프
  const mixer = new MixerGraph(ctx);
  const effects = new EffectRack(ctx, mixer, o.getEffects, function () { return o.bpm; });
  const rack = new InstrumentRack(ctx, function (id) { return mixer.trackInput(id); }, o.getInstruments,
function (sid) { return buffers[String(sid)] || null; });
  const player = new ClipPlayer(ctx, function (id) { return mixer.trackInput(id); }, function (sid) { re
turn buffers[String(sid)] || null; });
  const notePlayer = new NotePlayer(ctx, function (id, pitch, vel, when, dur) { return rack.playNote(id,
pitch, vel, when, dur); });
  effects.sync(o.tracks);
  mixer.applyTracks(o.tracks, true);

  // 3) 창마다 소리를 예약
  const noteEvents = expandNotes(midiClips).filter(function (ev) { return ev.abs_start >= o.startTick;
});
  const bounds = windowBoundaries(o.startTick, o.endTick, o.bpm, changes, WINDOW_SECONDS);
  const scheduleWindow = function (k) {
    const a = bounds[k];
    const b = bounds[k + 1];
    const nowTime = ticksToSecondsWithChanges(a, o.bpm, changes) - ticksToSecondsWithChanges(o.startTic
k, o.bpm, changes);
    if (k === 0) {
      // 시작 위치에 걸쳐 있는 클립은 중간부터 울림
      clipsCovering(audioClips, a).forEach(function (c) { player.schedule(c, a, nowTime, o.bpm, change
s); });
    }
    clipsStartingIn(audioClips, a, b).forEach(function (c) { player.schedule(c, a, nowTime, o.bpm, chang
es); });
    notesStartingIn(noteEvents, a, b).forEach(function (ev) { notePlayer.schedule(ev, a, nowTime, o.bpm,
changes); });
  };
  scheduleWindow(0);

  // 4) 창 경계마다 멈춰서 다음 창을 예약. 경계 시각은 렌더링 샘플 단위로 맞춰짐
  const t0 = ticksToSecondsWithChanges(o.startTick, o.bpm, changes);
  const stops = [];
  for (let k = 1; k < bounds.length - 1; k++) {
```

```
    const sec = ticksToSecondsWithChanges(bounds[k], o.bpm, changes) - t0;
    if (sec > 0 && sec < totalSec) stops.push({ k: k, sec: sec });
  }
  let failed = null;
  stops.forEach(function (s) {
    ctx.suspend(s.sec).then(function () {
      if (failed) { ctx.resume(); return; }
      if (cancelled()) { failed = new Error('cancelled'); ctx.resume(); return; }   // 남은 구간은 소리
없이 빠르게 끝내고 결과를 버림
      scheduleWindow(s.k);
      progress(0.1 + 0.9 * (s.sec / totalSec));
      ctx.resume();
    });
  });

  const rendered = await ctx.startRendering();
  if (failed) throw failed;
  progress(1);
  const channels = [];
  for (let c = 0; c < rendered.numberOfChannels; c++) channels.push(rendered.getChannelData(c));
  return { channels: channels, sampleRate: rendered.sampleRate, frames: rendered.length };
}
```

4. 바뀐 점

위치	바뀐 내용
`PLAYABLE_CODES`	`['synth_sub']`에서 `['synth_sub', 'sampler']`로 늘었습니다. 이 목록에 없는 악기는 노트를 받아도 아무 소리도 내지 않습니다.
`InstrumentRack` 생성자	네 번째 인자 `getBuffer(sampleId)`를 받습니다. 없으면 항상 `null`을 돌려주는 함수로 대신해서, 이전 호출(인자 세 개)도 그대로 동작합니다.
`InstrumentRack.playNote`	악기 코드가 `sampler`이면 `getBuffer`로 샘플을 찾아 `playSamplerNote`를, 아니면 `playSynthNote`를 부릅니다.
`PlaybackSession.start`	클립의 샘플 번호에 `samplerSampleIds`로 구한 번호를 더하고, 중복 없이 모두 불러온 뒤에 재생합니다.
`renderOffline`	같은 번호 모음을 디코딩하고, `buffers` 객체를 `getBuffer`로 넘깁니다.

5. 최신 문법으로 쓰면

```
// 재생 시작 때 불러올 샘플 번호 모으기를 최신 문법으로 쓴 모양
const clipIds = this.clips.filter((c) => c.sample_id != null).map((c) => c.sample_id);
const trackIds = deps.getTracks().map((t) => t.track_id);
const ids = [...new Set([...clipIds, ...samplerSampleIds(deps.getInstruments ?? (() => []), trackId
s)])];
await Promise.all(ids.map((id) => this._loadSample(id)));
```

전통 문법 (실제 코드)	최신 문법
`forEach` 안에서 `indexOf`로 중복을 걸러 `push`	`new Set`과 전개
`deps.getInstruments ? deps.getInstruments : function () { return []; }`	`deps.getInstruments ?? (() => [])`

전통 문법 (실제 코드)	최신 문법
`function (t) { return t.track_id; }`	화살표 함수 `(t) => t.track_id`

6. 왜 이렇게

선택	이유
샘플 저장소를 `InstrumentRack`이 갖지 않고 `getBuffer` 함수로 받습니다	재생 세션은 `buffers`를, 오프라인 렌더링은 자기 `buffers`를 씁니다. 저장소를 함수로 주입하면 `InstrumentRack`은 어느 쪽인지 몰라도 되고, 컨텍스트를 인자로 받는 기존 설계(S1-11a)와 같은 방식입니다.
노트가 울리기 전에 샘플을 모두 불러옵니다	노트를 울리는 시점에는 비동기로 기다릴 수 없습니다. 재생 시작 때 모든 샘플을 불러오고 시작하는 기존 클립 처리(290)와 같은 원칙입니다.
꺼진 층과 `sample_id`가 없는 층은 불러오지 않습니다	쓰지 않을 샘플을 내려받고 디코딩하는 것은 낭비입니다. 샘플이 없는 Sampler는 소리만 나지 않을 뿐 오류가 아닙니다.
샘플을 못 불러와도 재생과 내보내기는 계속합니다	한 파일 때문에 전체가 멈추지 않게 하는 기존 방침(290, 299)을 그대로 따릅니다. `getBuffer`가 `null`을 돌려주면 301이 조용히 건너뜁니다.
같은 샘플은 한 번만 불러옵니다	클립과 Sampler가 같은 샘플을 쓰거나, 여러 층이 같은 샘플을 써도 `ids`에서 중복을 걸러 한 번만 디코딩합니다. 시험 H에서 확인합니다.
인자를 맨 뒤에 선택으로 추가했습니다	기존 시험 9개(115건)가 `InstrumentRack`을 인자 세 개로 만들고 있습니다. 호출하는 쪽을 고치지 않아도 되게 해서 회귀가 없음을 보장했습니다.

7. 시험

- 위 `sampler_unit.mjs`의 `PLAYABLE_CODES`와 `InstrumentRack` 부분이 목록과 `getBuffer` 호출을 확인합니다.
- `render_sampler.mjs`의 G와 H가 샘플을 불러오지 못한 경우와, 같은 샘플을 쓰는 두 층이 샘플을 한 번만 불러오는 것을 확인합니다.
- `docs/handoff/s1-11c_tests/ui_sampler.mjs` (39건)가 실제 화면에서 재생과 내보내기를 확인합니다.
  - 재생: 샘플이 없으면 조용하고, 있으면 노트 시작 직후 소리가 나며, `oneshot`은 노트(0.5초)가 끝난 뒤에도 샘플이 끝까지 울리고, `classic`은 노트가 끝나고 Release가 지나면 조용합니다.
  - 내보내기: 같은 두 경우가 `renderOffline` 결과에서도 똑같이 나옵니다.
- 기존 시험(S1-11a 9개, S1-11b 3개)은 모두 그대로 통과했습니다. 시험 중 발견한 문제는 없었습니다.